

INDICE

Introducción a ASP.NET

Revisando Conceptos

¿Que Problemas resuelve .NET?

Beneficios del Marco de Trabajo .NET

Vistazo sobre ASP.NET

Primera Aplicación Web con Visual Studio

Los lenguajes en .NET

Creando Componentes en.NET

Compilación en tiempo de corrida y Ejecución

¿Qué es un Formulario Web?

Usando Formularios Web y Controles con Web Matrix

4. - Introducción a ASP.NET

En este capítulo vamos a repasar algunos conceptos que vimos en el capítulo 1, veremos que es ASP.NET, los lenguajes que soporta VS.NET y como se compilan los programas. Además veremos como crear componentes y finalmente como usar formularios y controles tanto en VS.NET como con Web Matrix.

4. 1. - Revisando Conceptos

En esta sección repasaremos conceptos relacionados con VS.NET y Web Matrix, el Marco de Trabajo y el runtime de lenguaje común.

4. 1. 1. - Poniendo todas las piezas juntas.

La plataforma .NET es el modelo de desarrollo con el cual el **software es independiente del sistema operativo, del dispositivo físico** y que permite acceder a los datos desde Internet.

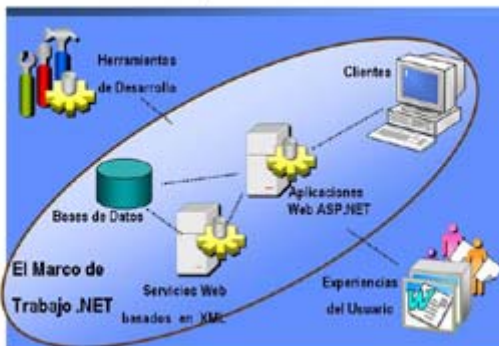
Con la plataforma .NET usted puede desarrollar la nueva generación de aplicaciones Windows y Web. El objetivo de la plataforma es simplificar el desarrollo. Está compuesta por los siguientes componentes:

- **El Marco de Trabajo .NET**
- Los Servidores .NET
- Servicios Web existentes
- Visual Studio .NET
- Web Matrix

El **Marco de Trabajo** tiene 2 elementos principales:

- **El Runtime de Lenguaje Común:** Administra servicios en tiempo de ejecución como por ejemplo, la seguridad, la administración de la memoria y la integración de los lenguajes.
- **Las librerías de Clases:** Proveen de código reusable para las tareas mas comunes como por ejemplo: Acceso a datos, Desarrollo de Servicios Web y formularios Web o Windows.

Juntando todas las piezas



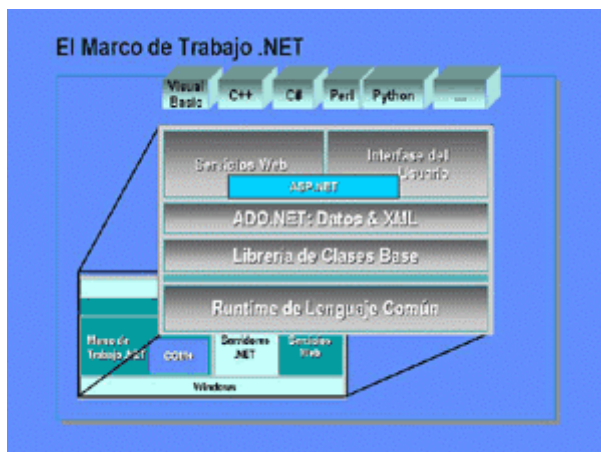
4. 1. 2. -¿Que Problemas resuelve .NET?:

El objetivo de .NET es eliminar varios de los problemas que se le presentan a los desarrolladores. Antes de .NET pocas aplicaciones se ejecutaban en más de una plataforma de hardware y de software por lo tanto debían reescribir el software para adaptarlo a los distintos entornos. Otro de los problemas era la comunicación y el intercambio de datos entre distintas aplicaciones.

La plataforma .NET resuelve estos problemas utilizando el Runtime de Lenguaje Común que es **independiente de hardware y software** y por medio de la utilización de XML como lenguaje universal de intercambio de datos entre aplicaciones. Ahora los desarrolladores pueden **escribir aplicaciones en cualquier lenguaje** .NET y estar seguros de que pueden ser ejecutadas en todas las plataformas de hardware y software compatibles con .NET. El Runtime de Lenguaje Común también se ocupa de problemas de infraestructura como por ejemplo el manejo de la seguridad y la memoria permitiéndole al desarrollador concentrarse en la lógica de su aplicación.

Los **servicios Web** basados en XML **utilizan XML para enviar datos**. Esto le permite a cualquier dispositivo de cliente que entienda XML recibir datos. XML es un lenguaje estándar. La mayoría de los clientes actuales pueden entender XML ejemplo de esto son: Las PC, los celulares, las consolas de juego o los handHelds.

4. 1. 3. - Repasando los componentes del Marco de Trabajo:



Marco de Trabajo provee de los cimientos tanto en tiempo de diseño como en tiempo de ejecución para construir y ejecutar aplicaciones .NET

Tiene distintos componentes:

- **Sistema Operativo:** El marco de trabajo debe correr sobre un sistema operativo. Actualmente toda la plataforma Windows permite la ejecución del Marco de Trabajo. Por ejemplo, Windows XP o Windows 2000 o Windows CE para dispositivos móviles

- **La librería de Clases:** Clases independientes de la plataforma que le simplifica la tarea a los desarrolladores. Estas clases pueden ser extendidas por los desarrolladores ya que estos pueden agregar sus propias clases.

- **El ejecutor de Lenguaje Común:** Provee servicios en tiempo de ejecución.

- **ADO.NET:** Son clases que proveen de acceso a los datos. Pueden entender XML.

- **ASP.NET:** Son clases que permiten construir los servicios Web y la interfase al usuario de las aplicaciones Web

- **Servicios Web:** Son componentes programables que pueden ser utilizados por las aplicaciones tanto en la Intranet como desde Internet. La plataforma .NET provee de herramientas para su creación, testeo y distribución.

- **Interfase del Usuario:** La plataforma soporta 3 interfases de Usuario. Consola de comando, Formularios Windows y Formularios Web.

Lenguajes: Cualquier lenguaje que sea compatible con la **Especificación de Lenguaje Común** puede ser utilizado en la plataforma .NET. Además de los lenguajes nativos de Visual Studio hay muchos otros por ejemplo PERL, Cobol o RPG.

4. 1. 4. - Beneficios del Marco de Trabajo .NET:

Beneficios del Marco de Trabajo

- Se basa en Estandares de Internet
- Cualquier lenguaje compatible con .NET lo puede usar
- El código está organizado jerárquicamente en espacios de nombres y clases
- Es un modelo sencillo y extensible

1960's 1990's 2000's

El desarrollo de aplicaciones evoluciona continuamente. Durante la década del 80 se hacían aplicaciones Windows usando APIs con el lenguaje C o C++. Después aparecieron lenguajes más específicos como VB o ASP. Con el Marco de Trabajo usted puede usar cualquier lenguaje compatible con la plataforma .NET

En la imagen usted tiene un resumen de los principales beneficios del Marco de Trabajo.

4. 1. 5. - Visual Studio: La herramienta para desarrollar aplicaciones .NET:

Visual Studio .NET: La herramienta .NET

Herramientas para Formularios Web Herramientas para Formularios Windows

Múltiples Lenguajes Manejo de Errores

Herramientas para Servicios Web Acceso a Datos

Diseño → Desarrollo → Depuración → Instalación

Visual Studio .NET es un entorno de desarrollo integrado pensado para hacer rápida y fácilmente las aplicaciones de la nueva generación

Asiste al desarrollador en las distintas etapas del ciclo de vida de una aplicación desde el diseño hasta la instalación. Como funciona sobre el Marco de Trabajo aprovecha sus ventajas lo que le permite entre otras cosas utilizar varios lenguajes.

4. 2. - Vistazo sobre ASP.NET:



Internet es la red de redes. Su evolución fue y es vertiginosa. El principal protocolo que utiliza es HTTP.

El contenido en su primera etapa era accedido mediante **páginas HTML estáticas**. HTML es un lenguaje de marcas. Estas marcas o 'TAGs' definen como se verá la información, por ejemplo, el color, o font o si se va a usar un cuadro para ordenar la presentación de la información.

Luego se incorporaron lenguajes como Java o VBScript que permitieron poner código en la página. HTML. **Este código era accedido por el cliente y se ejecutaba en la máquina del cliente**. Esto significó que las páginas tuvieran un contenido dinámico que podía cambiar, por ejemplo, una animación en el encabezado de una página. El siguiente paso fue el poner **código del lado del servidor** que provee y administra las páginas. La tecnología de Microsoft para esto fue ASP ejecutándose en el servidor 'Internet Information Services'.

Las **páginas ASP** permiten mezclar los TAGs HTML con código como Java o VBScript. Bajo este esquema cuando un usuario accede a las páginas HTML o ASP puede disparar la ejecución de código que se encuentra en páginas ASP. Esta "arma" da respuesta al cliente como una página HTML. Una ventaja del código del lado del servidor en las páginas ASP es la seguridad ya que el usuario no accede al código. Una desventaja es la performance ya que el código se compila en el momento de ser accedido. El siguiente paso de la evolución es **ASP.NET**.

4. 2. 1 - ¿Qué es ASP.NET?

¿Qué es ASP.NET?

- Es el sucesor de las páginas ASP.
- Son páginas Web dinámicas que pueden acceder a recursos del servidor.
- Permite la ejecución de las páginas Web del lado del servidor.
- Permite hacer Servicios Web.
- Es independiente del Explorador de Internet.
- Es independiente del Lenguaje de Programación.

ASP.NET es la tecnología que permite hacer **formularios Web y Servicios Web**.

Una aplicación Web ASP.NET puede estar formada por varios formularios Web.

El **formulario Web** es una página dinámica que puede acceder a los recursos del servidor o a otros servidores. Por ejemplo, una página ASP.NET puede ejecutar código para acceder a SQL Server y luego armar una respuesta al usuario como TAGs HTML.

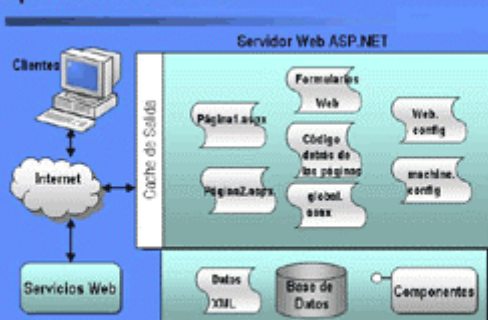
Como el código se ejecuta del lado del servidor, la respuesta puede adaptarse al Explorador de Internet del usuario.

Como ASP.NET esta construido sobre el Marco de Trabajo, su código puede ser escrito en cualquier lenguaje compatible con .NET.

Los **Servicios Web**, a los que le dedicaremos un capitulo, son hechos con la tecnología ASP.NET. Los Servicios Web son componentes que pueden ser accedidos desde la Intranet o desde Internet y permiten hacer aplicaciones distribuidas y centradas en el usuario.

4. 2. 2. - Aplicaciones Web ASP.NET

Aplicación Web ASP.NET



Las aplicaciones Web ASP.NET tienen varios componentes:

• **Formularios Web o páginas .ASPX:** Proveen de la **interfase visual**. No tienen código ejecutable.

• **Páginas de Código por detrás:** Están asociadas con cada formulario y son las que **proveen del código** ejecutable. A diferencia de las páginas ASP con la tecnología anterior, no se mezcla código y TAGs en la misma página.

• **Archivos de Configuración:** Son archivos que permiten configurar la aplicación, por ejemplo el archivo **Web.config** y el servidor por ejemplo el archivo **machine.config**.

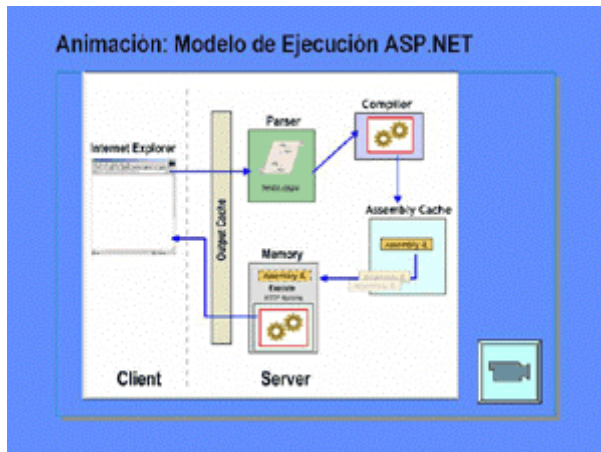
• **Global.asax:** Es un archivo que contiene código. Este código responde a eventos que se disparan en la aplicación Web.

• **Enlaces a Servicios Web XML:** Permite a la aplicación Web transferir datos XML desde y hacia Servicios Web

• **Conectividad a Bases de Datos:** Permite a la aplicación Web transferir datos desde y hacia distintas Bases de datos.

• **Caching:** Permite a la aplicación Web devolver formularios Web mas rápidamente después del primer acceso.

4. 2. 3. - El modelo de Ejecución ASP.NET:



• Cuando un cliente accede a una página Web por primera vez:

- El Explorador de Internet del cliente envía un pedido **GET HTTP**
- El analizador de ASP.NET interpreta el código fuente
- Si el código no fue compilado anteriormente como un DLL entonces ASP.NET ejecuta el compilador
- El Runtime carga y ejecuta el código intermedio **MSIL**

• Cuando un cliente accede a la misma página Web por segunda vez:

- El Explorador de Internet del cliente envía un pedido **GET HTTP**
- El Runtime carga y ejecuta inmediatamente el código **MSIL** que fue antes compilado durante la primera llamada del usuario.

4. 2. 4. - Recursos sobre ASP.NET:

Recursos sobre ASP.NET

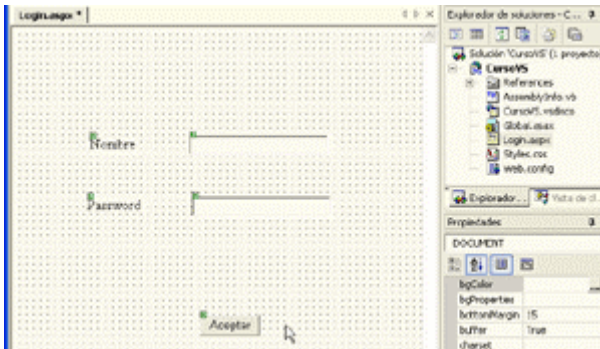
- **Documentación sobre el Marco de Trabajo .NET**
 - Código de Ejemplo
 - Tutoriales Rápidos
- **Comunidades en Internet.**
- **Sitios Web**
 - www.gotdotnet.com
 - www.ibuyspy.com
 - www.asp.net
 - msdn.microsoft.com
 - www.google.com

Usted tiene muchos lugares donde puede encontrar información adicional sobre ASP.NET en particular o sobre la plataforma .NET en general.

- Dentro de Visual Studio.NET apretando la tecla F1.
- En VS.NET la página de Inicio contiene los links a novedades, al sitio MSDN de Microsoft y a las comunidades en línea. Entre otros.
- En el sistema operativo yendo a Inicio, Programas, Microsoft .NET Framework SDK
- Consultando grupos de noticias en Internet
- Consultando sitios como:

- <http://www.asp.net.com> Sitio de Microsoft sobre ASP.NET. Tiene artículos, enlaces a otros sitios, foros de discusión y ejemplos.
- En <http://www.asp.net.com/webmatrix> El sitio oficial del proyecto ASP.NET Web Matrix
- <http://www.gotdotnet.com> . Un sitio muy exitoso y consultado. Tiene muchos artículos, ejemplos y Enlaces.
- <http://www.ibuyspy.com> . Es un sitio existente pero de una aplicación ficticia. Esta hecho para aprender ASP.NET (En Inglés)
- <http://msdn.microsoft.com> . Sitio en inglés de Microsoft para los desarrolladores o el sitio <http://msdn.microsoft.com/argentina> en castellano.
- <http://www.google.com>. Es un muy buen buscador de páginas.

4. 2. 5. - Primera Aplicación Web con Visual Studio



```
Dim strNombre As String = TextBox1.Text
Dim strPassword As String = TextBox2.Text
```

```
'Para el ejemplo: El nombre de usuario debe ser: curso
'La password deber ser: vs
'chequeo de Nombre
If strNombre <> "curso" Then
    Response.Write("Error, Nombre de Usuario Incorrecto")
Else
    'chequeo de Password
    If strPassword <> "vs" Then
        Response.Write("Error, Password Incorrecta")
    Else
        'va otra la página
        Response.Redirect("principal.aspx")
    End If
End If
```

Vamos a hacer una aplicación Web sencilla con VS.NET. Haremos una pantalla para captura del Nombre de Usuario y la password.

- Abra VS.NET.
- Cree un nuevo proyecto. Llámelo '**Aplicación Web**'
- En la pantalla de plantillas, seleccione '**Aplicación Web ASP.NET**'. Donde dice **ubicación** escriba '**http://localhost/CursoVS**'
- Haga click en **Aceptar**. Va a tardar un poco en abrir. Entre otras cosas lo que esta haciendo es crear un directorio virtual en su servidor de páginas Web llamado en este caso '**CursoVS**'
- Vaya al explorador de soluciones. Va a ver un formulario Web con extensión ASPX. Haga click sobre el una vez. Presione el botón derecho, Seleccione Cambiar Nombre, Escriba '**Login**'.
- Vaya al formulario, haga doble click sobre él para **abrir el editor de código**. Cambie el nombre de la clase por Login.
- Vaya a la vista diseño del formulario.
 - Agregue un control **Label**. Modifique la propiedad text por 'Nombre'
 - Agregue un control **Label**. Modifique la propiedad **Text** por 'Password'. Modifique la propiedad **TextMode** por 'password' para que cuando ingrese la password no se vea.
 - Agregue un control **TextBox**. Agregue otro control **TextBox**.
 - Agregue un control de tipo **Button**. Modifique la propiedad Text. Escriba 'Aceptar'
 - Haga **doble click en el control Button** que agregó. En el editor de código escriba el siguiente código.

- Ejecute el programa. Ingrese como nombre de Usuario: curso y como Password: vs. Haga click en el botón. Haga distintos intentos usando otro nombre de usuario y Password.
- Cierre la ventana.
- Salve la aplicación
- Salga de VS.NET

4. 2. 6. - Archivos de una Solución Web en Visual Studio.



Cuando usted crea una **solución** Web se generan varios archivos. Veamos para que sirve cada uno:

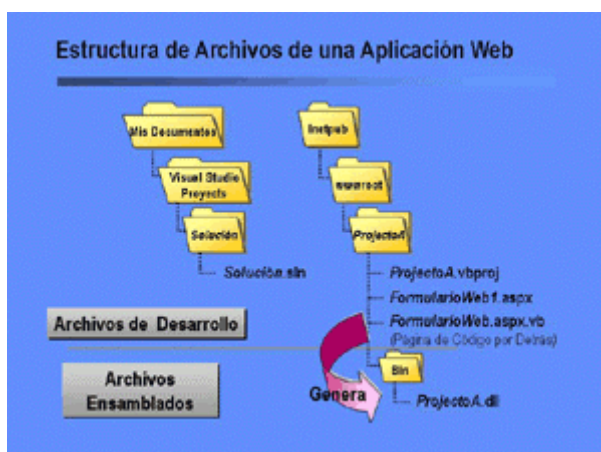
- **Archivo de la solución (.sln):**
 - Por default se guarda en el directorio \Mis documentos \ Visual Studio Projects.
 - Guarda los distintos proyectos que forman parte de una solución
- **Opciones de la solución del usuario (.suo):**
 - Guarda por ejemplo marcas de depuración o la configuración del entorno.

Las aplicaciones Web se guardan en un subfolder del directorio **inetpub\wwwroot**. Este es el directorio donde el servidor de páginas Web de Microsoft '**Internet Information Services**' (IIS) tiene los archivos. IIS genera en forma automática cuando usted crea un proyecto Web, un **directorio virtual** con el nombre que usted seleccionó como nombre de proyecto.

Tenga en cuenta que una solución puede estar formada por varios Proyectos, por ejemplo, dentro de una solución usted puede tener un proyecto Web y un servicio Web. Esto es algo que vamos a probar más adelante.

Veamos para que sirven los archivos de un **Proyecto Web**

- **Formulario Web ASP.NET (.aspx):** Es la interfase visual de la aplicación Web. Puede tener código por detrás., cuyo nombre es Formulario.aspx.vb o Formulario.aspx.cs
- **Servicio Web ASP.NET (.asax):** Es un servicio Web. Puede tener código por detrás con el nombre de ServicioWeb.asax.vb o ServicioWeb.asax.cs
- **Clases y código por detrás (.cs y .vb):** Son las clases que utiliza el proyecto y el código de soporte de los formularios y los servicios.
- **Archivos de Descubrimiento (.disco y .vsdisco):** Son archivos XML que contienen enlaces a recursos para proveer de información acerca de los servicios Web. **Permiten encontrar servicios Web en forma programática.**
- **Clase Global (.asax):** Es un archivo que contiene código de eventos a nivel de aplicación.
- **Web.config:** Es un archivo XML con información de configuración.
- **Archivos ensamblados del proyecto (.dll):** Todas las paginas de código por detrás de una aplicación son compilados en un solo DLL que se guarda en el directorio **/bin** con el nombre de **NombreDeProyecto.dll**
- **VsProj o CsProj:** Vs Proj es la extensión de un proyecto si usted escribió la aplicación en Visual.NET. CsProj es la extensión si utilizo c#



4. 3. - Los lenguajes en .NET

4. 3. 1 - Soporte de Múltiples Lenguajes.

Soporte de Múltiples Lenguajes

- **EL Marco de Trabajo esta pensado para soportar muchos lenguajes**
 - Actualmente hay más de 20
 - Microsoft provee de Visual Basic .NET, C#, Visual J# .NET, y JScript .NET
- **Beneficios por soportar muchos lenguajes**
 - El código de los módulos es más reusable
 - El acceso a las API es igual para todos
 - Se utiliza el lenguaje más adecuado para cada tarea
 - La performance es casi igual en todos los lenguajes

El Framework de .NET fue pensado para no poner límites a la cantidad de lenguajes que puede utilizar.

La imagen resume las principales ventajas que tiene .NET al soportar varios lenguajes

4. 3. 2. - El Runtime de Lenguaje Común.

El Runtime de Lenguaje Común

- **Un Runtime para todos los Lenguajes .NET**
- **Administra los Hilos y la Memoria**
 - Recolección de Basura
- **Administra la Seguridad**
- **Elimina los problemas de versiones con los DLL**
 - Múltiples versiones de un DLL pueden correr al mismo tiempo
 - Las aplicaciones pueden especificar que versión de un DLL usan

El Runtime de Lenguaje Común es el motor del Marco de Trabajo. Provee de varios servicios, como por ejemplo la administración de la memoria, la seguridad y el manejo de versiones:

- Antes de .NET los **DLL** que utilizaban las aplicaciones eran compartidos por default. Por lo tanto, al instalar un DLL modificado para una aplicación otras aplicaciones podían dejar de funcionar.

- Ahora se puede indicar mediante archivos de configuración que versión de un DLL se usa para cada aplicación. Con esto se evita lo que se conoce como "El infierno de los DLL".

4. 3. 3. - Compilación en tiempo de corrida y Ejecución



En la imagen puede ver lo que sucede cuando una página Web es accedida por el usuario

Cada compilador de un lenguaje compatible con .NET compila el código de un programa a MSIL. MSIL es código intermedio de alto nivel y es independiente del hardware.

Después de que el código se convierte a MSIL el Runtime de Lenguaje Común lo transforma a código de máquina y entonces la aplicación se ejecuta.

4. 3. 4. - Que son los Espacios de Nombres

¿Qué son los Espacios de Nombre?

- **Son Grupos de Clases Relacionadas**
 - Es una agrupación Lógica no Física
 - Los Espacios de Nombre son jerárquicos
- **Decrece el conflicto de nombres**
- **Palabra clave Imports en Visual Basic .NET**

```
Imports System.Data.SqlClient
```

Son una agrupación lógica y jerárquica de clases que pueden ser usadas por cualquier lenguaje dentro de .NET. A cada clase dentro de un espacio de nombres se la llama tipo.

Hay 2 maneras de hacer referencias a un tipo:

- **Espacio de Nombre + Tipo**
- Usando la sentencia **Imports** al comienzo de un programa para evitar repetir continuamente el espacio de nombres al hacer uso de un tipo. Después en el código se usa directamente el nombre del tipo sin necesidad de calificarlo con el Espacio de Nombres.

En la figura tiene un ejemplo:

La sentencia **Imports** evita tener que calificar a las clases dentro del espacio de nombres de **System.Data.SqlClient** cada vez que se hace referencia a ellas en un programa.

4. 3. 5. - Usando los Espacios de Nombres:

Usando Espacio de Nombres

■ Declaración de Objetos Implícita

```
Imports System.Web.UI.WebControls  
...  
Dim listBox1 As New ListBox()  
listBox1.Items.Add("Primer Item")
```

■ Declaración de Objetos Explícita

```
Dim listBox1 As New System.Web.UI.WebControls.ListBox()  
listBox1.Items.Add("Primer Item")
```

En la imagen tiene 2 ejemplos de declaración de objetos.

La forma **Implícita no requiere del espacio de nombre** de la clase a partir de la cual el objeto se instancia.

La forma **Explícita requiere calificar la clase** del objeto con el espacio de nombre porque no se hace uso de la sentencia **Imports**.

4. 3. 6. - Comparando los distintos Lenguajes de VS.NET:

Seleccionando un Lenguaje

- La librería de clases del Marco de Trabajo .NET es la misma independientemente del Lenguaje
- Performance
 - Todos los lenguajes se compilan a MSIL
 - La única diferencia es como cada compilador de un lenguaje compila a MSIL
 - El runtime compila de la misma manera MSIL sin importar cual es el lenguaje de origen
- Experiencia de Desarrollo
 - C# es parecido a Java, C, Visual C++, y Pascal
 - Visual Basic .NET es parecido a Visual Basic
- Compatibilidad con los Exploradores de Internet
 - El código ASP.NET es código del lado del Servidor, por lo tanto la compatibilidad con los Exploradores de Internet no es problema

Como todos los lenguajes son compilados a código intermedio MSIL no hay muchas diferencias entre los lenguajes. La selección de un lenguaje depende principalmente de la experiencia previa.

Es decir, a diferencia de la plataforma de desarrollo anterior, casi no hay diferencias de performance entre los lenguajes.

En la imagen usted puede ver una comparación entre los distintos lenguajes

4.4. - Creando Componentes en .NET:

En esta sección vamos a ver como crear clases y componentes y veremos un ejemplo de su uso en páginas Web ASP.NET.

4.4.1. - ¿Qué son las Clases y los Componentes?

Las **clases** son una manera de abstraer y agrupar funcionalidad mediante métodos y propiedades en los lenguajes Orientados a Objetos como Visual Basic.NET y C#

Las clases permiten abstraer o encapsular la funcionalidad porque cuando un objeto es generado a partir de una clase, el programador no necesita saber como están implementados los métodos de esa clase. Lo que le interesa es lo que estos métodos le permiten hacer. Por ejemplo, cuando usted utiliza el método `MessageBox.Show ()` para mostrar algo en una aplicación Windows, usted no necesita saber como esta implementado. Solamente lo utiliza.

En resumen:



Las clases son un conjunto de código que no tiene interfase visual.

Este código puede ser compartido por distintas partes de una aplicación. Permiten organizar funcionalidad

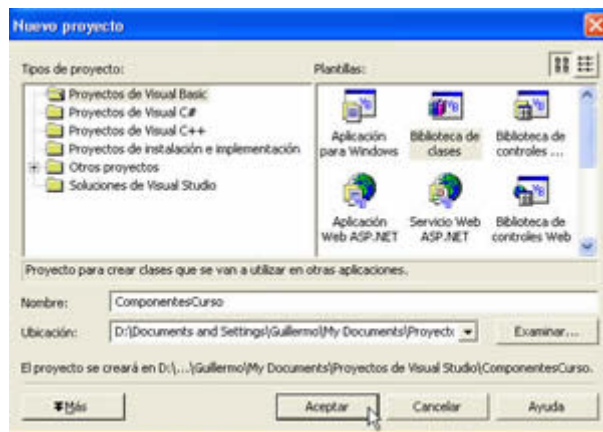
- Las clases pueden tener propiedades y métodos.
 - Las **propiedades** permiten configurar las características de una clase.
 - Los **métodos** son una manera de invocar las acciones que una clase provee.
- Los **Objetos son instancias de clases** que se generan a partir de las plantillas que las clases proveen.
- Los **Componentes son clases compiladas en un DLL**. Debido a que el componente es un archivo, su funcionalidad puede ser accedida por distintas aplicaciones. Usted hace referencia a un componente desde una aplicación usando el espacio de nombre de un componente y el nombre de la clase.

4. 4. 2.A - Creando una Clase en Visual Studio .NET

Usted puede **crear una clase en un proyecto de .NET**. También puede crear una **librería de Clases**.

Cuando usted **crea una clase dentro de un proyecto** la clase existe dentro del espacio de nombre del proyecto. La clase es compilada dentro del proyecto y **no puede ser reutilizada por otras aplicaciones**.

Cuando usted **crea una clase en una librería de clases** VS.NET crea un espacio de Nombres default. El archivo de la clase se recompila como un DLL y por lo tanto se **crea un componente que permite su reutilización por otras aplicaciones**



Vamos a crear una Librería de Clase para generar un componente.

- Abra VS.NET
- Cree un **Nuevo Proyecto**
 - Vaya a Archivo, Nuevo, Proyecto
- Seleccione la plantilla llamada **'Biblioteca de Clases'**
- Haga click en **Aceptar**
- Vaya al explorador de soluciones.
- Haga click en el archivo llamado **class1.vb**
- Presione el botón derecho del Mouse
- **Renombre el archivo**. Escriba **Beneficios**
- Vaya al editor de código de la clase Beneficios
- Escriba el siguiente código

Public Class Beneficios

Structure InfoBeneficios

Dim strNombre As String

Dim strPagina As String

End Structure

'Observe que el metodo devuelve un arreglo de InfoBeneficios.

'Preste atención a los simbolos '()' en el tipo de datos

'InfoBeneficios

Public Function ObtenerBeneficiosList() As InfoBeneficios()

'Devuelve un arreglo de 2 dimensiones con los Beneficios

'Disponibles y los nombres de

'las paginas Web ASP.NET que implementan los beneficios

Dim arBeneficios(3) As InfoBeneficios

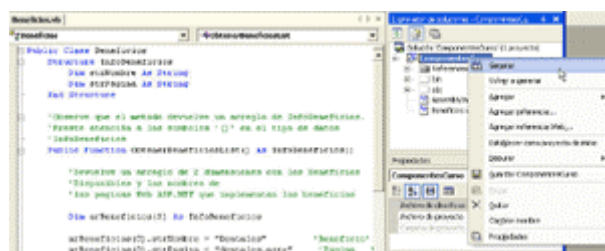
```
arBeneficios(0).strNombre = "Dentales"
arBeneficios(0).strPagina = "dentales.aspx"
arBeneficios(1).strNombre = "Medicos"
arBeneficios(1).strPagina = "medicos.aspx"
arBeneficios(2).strNombre = "Seguro de Vida"
arBeneficios(2).strPagina = "vida.aspx"
arBeneficios(3).strNombre = "Retiro"
arBeneficios(3).strPagina = "retiro.aspx"
```

```
'Beneficio'
'Pagina '
'
'
'
'
'
'
```

Return arBeneficios

End Function

End Class



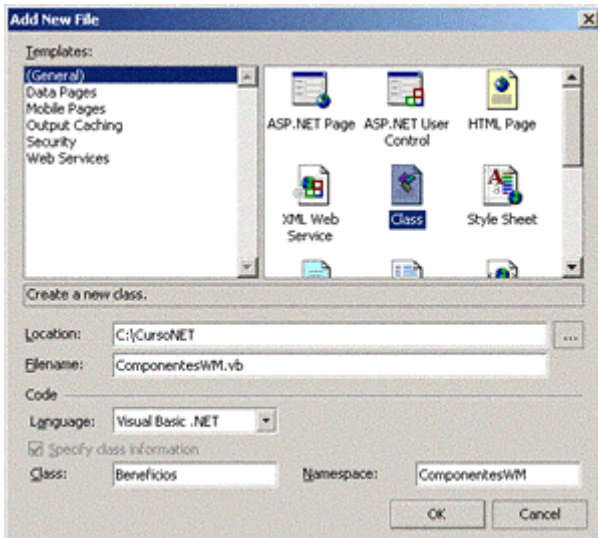
La clase tiene un tipo de datos **Estructura** llamado **InfoBeneficios**, que tiene 2 elementos Nombre y Página.

También expone un método llamado **ObtenerBeneficios** este método devuelve un arreglo de InfoBeneficios de 4 posiciones

- Ahora vamos a compilar el componente.
- o Vaya al explorador de soluciones.
- o Haga click en el nombre del Proyecto '**ComponentesCurso**'
 - Presione el botón derecho del Mouse
 - Selecione Generar
- Salve el proyecto
- **Cíérrelo**. Para eso vaya a **Archivo, Cerrar**

Acaba de generar un componente. Es un archivo llamado **ComponentesCurso.dll**. Búsquelo en su disco rígido debe estar en un folder llamado 'Mis **Documentos\Visual Studio Projects\ComponentesCursolbin**'

4. 4. 2.b. - Creando una Clase con Web Matrix.



Vamos a crear una librería de clases en **Web Matrix**.

- Abra **Web Matrix**, el cuadro de diálogo **Add New File** aparecerá
- Seleccione **(General)** y **Class** como se muestra en la figura.
- En **FileName** coloque ComponentesWM
- En **Language** mantenga el valor de Visual Basic .NET
- En **Class** coloque Beneficios y en **Namespace** coloque ComponentesWM y Pulse OK.

```
' ComponentesWM.vb
Imports System
Namespace ComponentesWM
    Public Class Beneficios
        Public Sub New()
        End Sub
        Structure InfoBeneficios
            Dim strNombre as String
            Dim strPagina as String
        End Structure
        Funcion que devuelve un arreglo de infoBeneficios
        Public Function ObtenerBeneficiosList() as InfoBeneficios
            dim arBeneficios(3) as InfoBeneficios
            arBeneficios(0).strNombre = "Dentales" 'Beneficio
            arBeneficios(0).strPagina = "dentales.aspx" 'Pagina
            arBeneficios(1).strNombre = "Medicos"
            arBeneficios(1).strPagina = "medicos.aspx"
            arBeneficios(2).strNombre = "Seguro de Vida"
            arBeneficios(2).strPagina = "vida.aspx"
            arBeneficios(3).strNombre = "Retiro"
            arBeneficios(3).strPagina = "retiro.aspx"
            Return arBeneficios
        End Function
    End Class
End Namespace
```

Edite el código, como se muestra a continuación:

La clase tiene un estructura llamada **infoBeneficios**, que está compuesta por **strNombre** y **strPagina**, que representa tanto el beneficio como la página de dicho beneficio. La función **ObtenerBeneficiosList** devuelve el arreglo de **infoBeneficios**.

- Guarde el Archivo.

Nota: A diferencia de **Visual Studio .NET**, **Web Matrix** no compila automáticamente los archivos de clase en assemblies. Esto se tiene que hacer mediante líneas de comando con el compilador **vbc.exe**

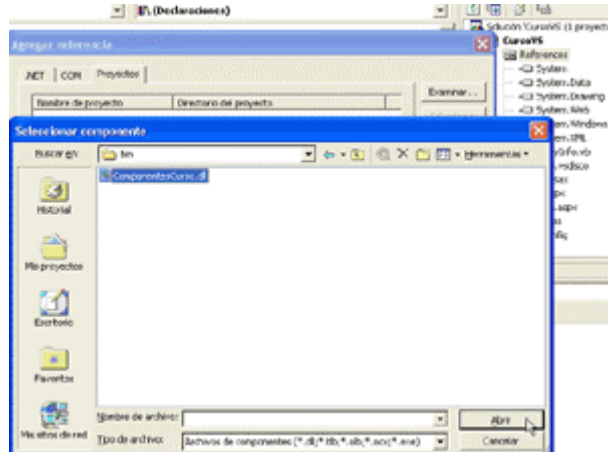
- Ejecute el modo comando de Windows, mediante **Inicio > Ejecutar > Escriba cmd**. También puede utilizar Visual Studio .NET Command Prompt.

- Cambie el directorio por **C:\WINDOWS\Microsoft.NET\Framework\v1.0.3705** que es donde se encuentra vbc.exe y escriba el siguiente comando:

```
vbc /t:library /out:C:\cursoNET\ComponentesWM.dll c:\cursoNET\ComponentesWM.vb
```

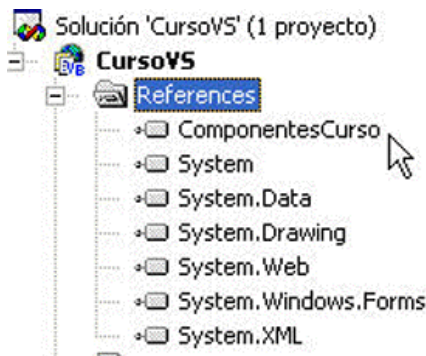
Compilará el componente **ComponentesWM.vb** de tipo library y lo pondrá en c:\cursoNET. Con eso ya creamos y compilamos un componente con **Web Matrix**. Puede verificar si se creó el archivo componentesWM.dll en la carpeta c:\CursoNET.

4. 4. 3.A. - Usando Componentes en ASP.NET:



Ahora vamos a usar el componente dentro de una aplicación Web.

- Abra el Proyecto Web anterior llamado 'CursoVS'
- Abra el explorador de soluciones
- Haga click donde dice 'references'
- Presione el botón derecho del Mouse
- Seleccione 'Agregar referencia'
- Seleccione examinar. Busque el archivo ComponentesCurso.dll.
- Seleccione Abrir
- Seleccione Aceptar. Ahora su proyecto tiene una referencia al componente que hizo el la sección anterior.
- Salve el proyecto



Usando componentes en un Formulario Web ASP.NET

- Agregue una referencia al DLL
- Inicialice el objeto:

```
Dim x As New ComponentesCurso.Beneficios
```

component.dll

```
Namespace ComponentesCurso
Class Beneficios
Function ObtenerBeneficios (...)
End Class
End Namespace
```

- Use el Objeto:

```
Lista = x.ObtenerBeneficios()
```

En la imagen tiene un resumen de cómo utilizar un objeto a partir de un componente.

Observe la diferencia entre el espacio de nombres que es **ComponentesCurso** y el nombre de la clase, en este caso **Beneficios**.

Preste atención también a la manera con que se inicializa un objeto usando la palabra **New**.

4. 4. 3.b. - Usando componentes en ASP.NET con Web Matrix.

Ahora utilizaremos el componente que creamos en el punto 4.4.2.b.

- Abra **Web Matrix** si no lo tiene abierto, cree un nuevo proyecto de tipo ASP.NET Page, coloque en Location c:\CursoNET o el directorio en el que trabajo para crear la clase, luego Pulse OK
- Vaya al directorio c:\cursoNet y cree una nueva carpeta llamada **bin**. Copie ahí el archivo **componentesWM.dll**, es ahí por defecto donde busca los componentes.
- Coloque **Web Matrix** en vista diseño y arrastre un control Button a la página.
- A continuación coloque la vista **All**, y edite el código como se muestra a continuación:

```
C:\CursoNET\Cliente.aspx *
1  <%@ Page Language="VB" %>
2  <%@ import Namespace="ComponentesWM" %>
3  <script runat="server">
4
5      ; Insert page code here
6
7
8      Sub Button1_Click(sender As Object, e As EventArgs)
9          dim x as new ComponentesWM.Beneficios()
10         dim y as object
11         y = x.ObtenerBeneficiosList()
12     End Sub
13
14 </script>
15 <html>
16 <head>
17 </head>
18 <body>
19     <form runat="server">
20         <asp:Button id="Button1" onclick="Button1_Click" runat="ser
21         <!-- Insert content here -->
22     </form>
23 </body>
24 </html>
```

La línea **<%@ import Namespace="ComponentesWM" %>** es la que hace la referencia al componente, para luego pueda ser utilizado.

4. 5. - Creando Formularios Web ASP.NET

4. 5. 1. - ¿Qué es un Formulario Web?

¿Qué es un Formulario Web?

- Tiene extensión .aspx
- Atributos de Página
 - Directiva @ Page
- Atributos en Body
- Atributos en Form

```
<%@ Page Language="vb" Codebehind="WebForm1.aspx.vb"
SmartNavigation="true"%>
<html>
<body ms_positioning="GridLayout">
<form id="Form1" method="post" runat="server">
</form>
</body>
</html>
```

Un formulario Web es una mezcla de Tags HTML, código y controles que se ejecutan en el servidor.

El cliente recibe código HTML, el servidor procesa el código y arma la pagina HTML que ve el cliente.

La extensión de un formulario Web es .aspx.

Los Formularios Web están formados comúnmente por 2 archivos.

- **.aspx** : Contiene los tags HTML de la interfase al usuario
- **.aspx.vb**: Es el llamado código por detrás. Este es el código que agrega funcionalidad a la página.

4. 5. 2. - Usando Controles del lado del Servidor.

En esta sección veremos los controles del lado del servidor

4. 5. 2. 1. - ¿Qué es un Control del lado del servidor?

¿Qué es un control del Lado del Servidor?

```
<asp:Button id="Button1" runat="server"
Text="Submit" />
```

- Runat="server"
 - Los Eventos se ejecutan en el servidor
 - Se guarda información del estado del Control
- Adaptan el código de acuerdo al tipo de Explorador

Son componentes que se ejecutan en el servidor y encapsulan la interfase del usuario y otra funcionalidad asociada.

Ejemplos de estos controles son:
Botones, Casillas de Texto y listas desplegables.

Observe la figura, la sintaxis runat="server" indica donde se ejecuta el código.

En este tipo de controles se puede guardar información sobre el estado del control a diferencia de los controles HTML estándares.

También pueden adaptar el código que envían al cliente de acuerdo al tipo de explorador que utiliza el usuario

4. 5. 2. 2. - Tipos de Controles

Tipos de Controles del Lado del Servidor

- **Controles HTML**
- **Controles Web**
 - Controles Intrínsecos
 - Controles de Validación
 - Controles Ricos
 - Controles asociados a Listas
 - Controles del Explorador de Internet

Hay muchos tipos de controles en ASP.NET.

Algunos son nuevos y otros son parecidos a los controles HTML. Se los puede agrupar en distintas categorías:

- **Controles Intrínsecos:** Se los utiliza para reemplazar a elementos HTML simples como por ejemplo una lista HTML
- **Controles de Validación:** Permiten con muy poco código controlar el ingreso de datos.
- **Controles ricos:** Son controles que implementan una funcionalidad compleja como por ejemplo un calendario
- **Controles Asociados a Listas:** Permiten mostrar datos en forma de lista dentro de una página ASP.NET
- **Controles del Internet Explorer:** Son controles complejos. Permiten implementar, por ejemplo, por ejemplo la barra de tareas en una página.

4. 5. 2. 3. - Guardando el Estado

Salvando el ViewState (Estado de la Vista)

- **Control ViewState oculto de pares de valores Nombre-Valor guardados en el formulario Web**

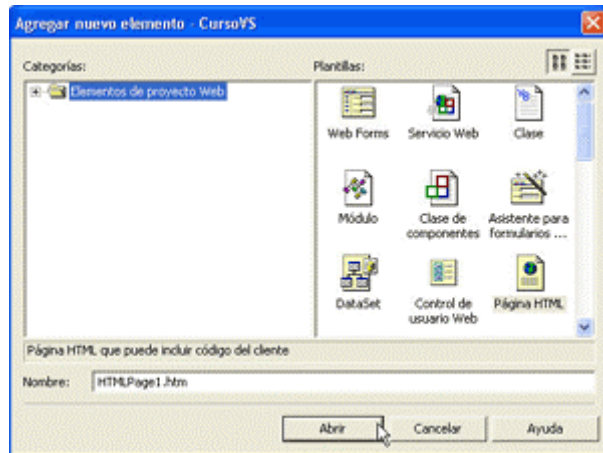
```
<input type="hidden" name="__VIEWSTATE"
value="dDxtMTA4MzE0MjEwNTs7Pg==" />
```

- **Habilitado por default, puede ser ajustado a nivel de formulario y a nivel de control**

```
<%@ Page EnableViewState="False" %>
<asp:ListBox id="ListName"
EnableViewState="true" runat="server">
</asp:ListBox>
```

Un problema en los sitios es como guardar el estado de los controles, por ejemplo la selección del usuario o como esta configurado un control. Esto se debe a que la página viaja continuamente entre el cliente y el servidor. Los formularios ASP.NET guardan el estado de los controles en la página usando un control oculto llamado **ViewState**. A medida que una página viaja entre el cliente y el servidor, su estado puede ser actualizado en cualquiera de los 2 lados y se guarda en el control **_ViewState**. Este control tiene una lista con el nombre de cada control en la página y su respectivo valor.

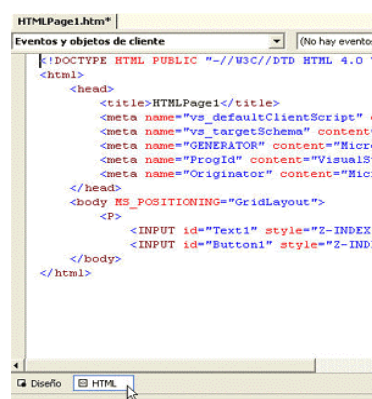
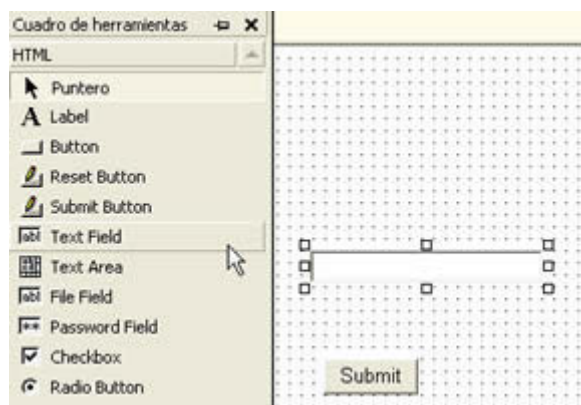
4. 5. 2. 4. - Práctica : Convirtiendo una página HTML a ASPX con Visual Studio .NET.



Vamos a agregar una página HTML a nuestro proyecto y a convertirla en ASPX. También modificaremos los controles HTML para que se ejecuten no desde el cliente sino del lado del servidor.

- Abra el proyecto anterior llamado '**CursoVS**'
- Vaya al explorador de soluciones.
- Haga click en el nombre del proyecto '**Curso VS**'
- Presione el botón derecho del Mouse
- Seleccione Agregar, Agregar Nuevo Elemento. Va a aparecer una plantilla.
- Seleccione Página HTML.
- Haga click sobre la página en el explorador de soluciones.

- Agregue un control '**Text Field**'
- Agregue un control '**Submit Button**'



- Haga click en la solapa que dice HTML, esta se encuentra en el borde inferior de la página. Observe el código HTML de la página que se encuentra entre el tag **<body>** y **</body>**.

Haga click en la imagen para expandir

Cap. 4 – Pág. 22

Data Markets

noticiasMUG

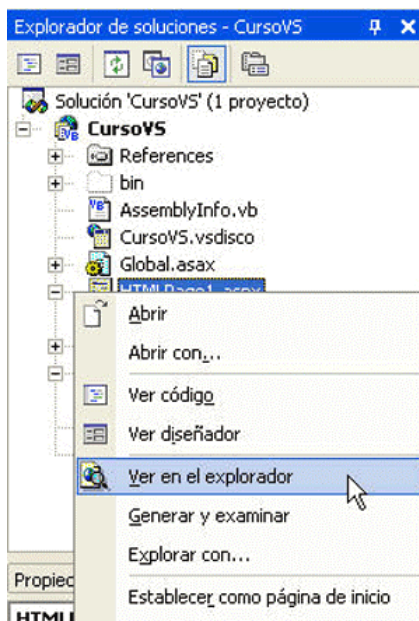
bumeran™
Soluciones tecnológicas de RRHH

UTN

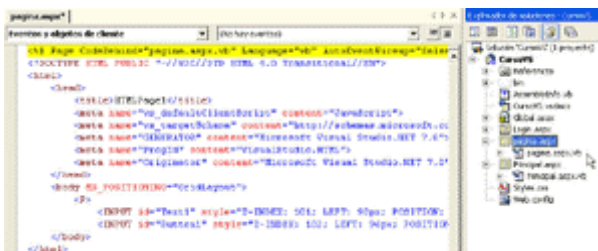
Microsoft .net

```
</head>
<body MS_POSITIONING="G:
<form >
  <INPUT id="Text1" s:
  <INPUT id="Submit1"
</form>
</body>
</html>
```

- Agregue el tag `<form>` y `</form>` tal como aparece en la figura



- Vaya al explorador de soluciones. Haga click en la página HTML. Presione el botón derecho del Mouse. **Modifique la extensión** de la página por **.aspx**.

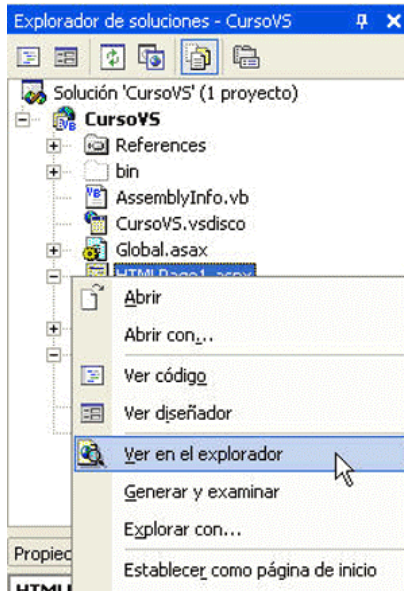


- Observe la ventana de Código HTML de la página. Vea como se agregó código, por ejemplo,

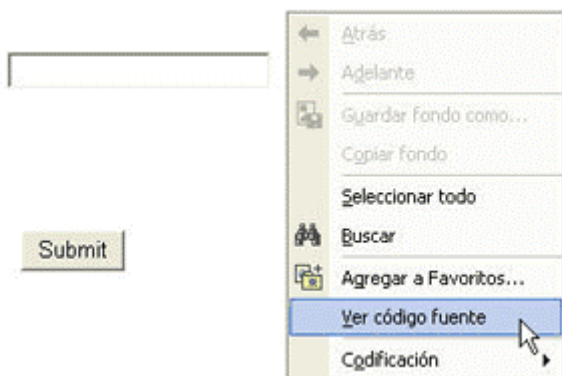
`<% Page CodeBehind=pagina.aspx.vb %>`

```
<body MS_POSITIONING="GridLayout">
  <form runat=server >
    <INPUT id="Text1" runat="server" st
    <INPUT id="Submit1" runat="server" st
  </form>
</body>
</HTML>
```

- Agregue el atributo **runat=server** al tag `<form>`, y a `text1` y `Submit1` como aparece en la imagen



- En el explorador de soluciones haga click sobre la página. Presione el botón derecho del Mouse, seleccione '**Generar y examinar**'



- Vaya a la página que aparece y presione el botón derecho del Mouse. Seleccione '**Ver Código fuente**'

```
<body ID_POSITIONING="GridLayout">
  <form name="ct10" method="post" action="HTM1.aspx" id="ct10">
    <input type="hidden" name="__VIEWSTATE" value="d3o3MgqHT:5K1g7Cz6LYdHTVM6VV"
    <input name="Text1" id="Text1" type="text" style="Z-INDEX: 101;
    <input name="Submit1" id="Submit1" type="submit" style="Z-INDEX:
  </form>
</body>
</HTML>
```

Observe que se agregaron automáticamente atributos al tag **<form>**. También se agregó un control oculto cuyo nombre es **__VIEWSTATE**. Este es el control que guarda el estado de los controles.

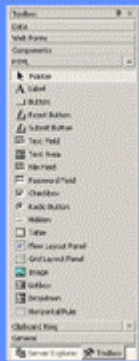
- Salve y cierre el proyecto

4. 5. 2. 5. - Controles HTML del Servidor

Controles HTML del lado del Servidor

- Basados en elementos HTML
- Existen dentro del espacio de nombres System.Web.UI.HtmlControls

```
<input type="text" id="txtName"
runat="server" />
```



Tal como vimos en el ejemplo anterior los controles HTML pueden ser del lado del cliente o del lado del servidor si se agrega el atributo **runat=server**

4. 5. 2. 6. - Controles Web del Servidor

Controles Web del lado del Servidor

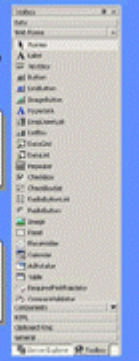
- Existen dentro del espacio de nombres System.Web.UI.WebControls namespace

Sintaxis del control

```
<asp:TextBox id="TextBox1"
runat="server">Text_to_Display
</asp:TextBox>
```

HTML generado por el control

```
<input name="TextBox1" type="text"
value="Text_to_Display"
Id=" TextBox1" />
```

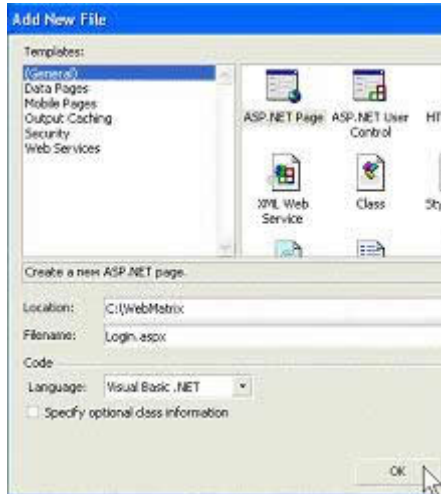


Los controles Web del lado del servidor fueron creados específicamente para ASP.NET. Algunos son bastante complejos como por ejemplo el control 'Calendar'. Este componente es muy sencillo de usar y permite ver un calendario y seleccionar fechas.

Observe en la imagen la diferencia entre el código que usted escribe en la página y el código HTML que se genera para el control.

4. 5. 2. 7. - Seleccionando el control adecuado

4. 6. - Usando Formularios Web y Controles con WebMatrix:



Vamos a crear un Formulario sencillo con WebMatrix.

- Cree un directorio llamado **c:\WebMatrix**. Lo usaremos para guardar los archivos del proyecto.
- Vaya a Inicio, Programas, Microsoft ASP.NET Web Matrix. Seleccione ASP.NET Web Matrix.
- Va a ver una ventana con plantillas. Deje la opción default '**ASP.NET Page**'. En la casilla '**Location**' escriba **c:\WebMatrix**. En la casilla '**filename**' escriba '**Login.aspx**'. Haga click en **OK**.

Va a ver el entorno de desarrollo de WebMatrix.

- Vaya a la ventana llamada '**Workspace**'. Expanda los directorios tal como se ve en la figura.

A diferencia de VS.NET no tiene un explorador de soluciones. Usted esta accediendo a un directorio donde tiene los archivos.

Ahora vamos a agregar 2 botones 'Labels', 2 botones 'TextBox' y un 'Button'.

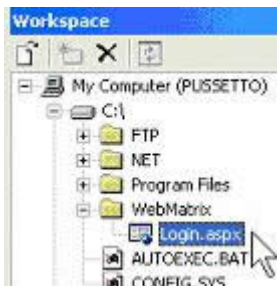
- Vaya al formulario. Observe en los tabs en la parte inferior izquierda de la ventana. Tiene 4 vistas.
 - **Diseño**: Para agregar los controles.
 - **HTML**: Para ver el Código HTML.
 - **Code**: Para ver el código VB.NET (Si selecciono VB en la plantilla de inicio de WebMatrix, el otro lenguaje posible es C#).
 - **All**: Ve la página completa con tags y código.

- Posicione el cursor donde quiere poner el primer control '**Label**'. Use la barra espaciadora y la tecla "enter" para esto.
- Vaya a la ventana llamada '**ToolBox**' tome un control de tipo '**Label**' y suéltelo donde quiere dejarlo.

- Haga click sobre el control '**Label**' y vaya a la ventana llamada '**Properties**'. Modifique la propiedad "text" por '**Nombre**'.

- Nuevamente haciendo uso de la barra espaciadora y la tecla "enter" posicione el cursor para dejar la pantalla tal como se ve en la figura. Modifique la propiedad Text de los controles para tal fin.

- Haga click en la casilla de Texto a la derecha de '**Password**'



Cap. 4 – Pág. 26

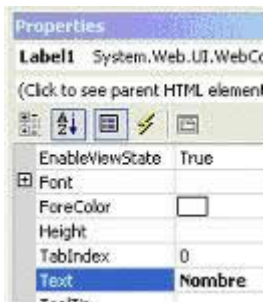
DataMarkets

noticiasMUG

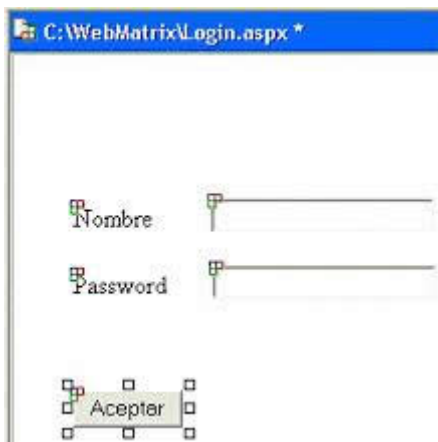
bumeran™
Soluciones tecnológicas de RRHH

UTN

Microsoft
.net



y vaya a la ventana **'Properties'**. Modifique la propiedad TextMode. Escriba **'Password'**.

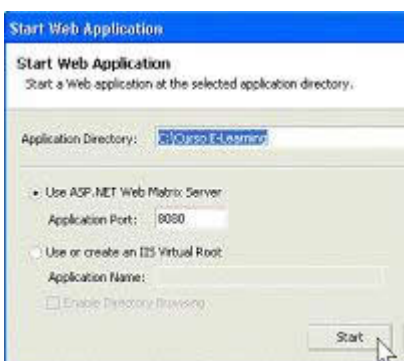


• Nuevamente haciendo uso de la barra espaciadora y la tecla “enter” posicione el cursor para dejar la pantalla tal como se ve en la figura. Modifique la propiedad Text de los controles para tal fin.

• Haga click en la casilla de Texto a la derecha de **'Password'** y vaya a la ventana **'Properties'**. Modifique la propiedad TextMode. Escriba **'Password'**



• Vamos a probar lo que hicimos. Vaya a la barra de herramientas y haga click en el icono de la figura.
• Va a aparecer una ventana para seleccionar el servidor Web de la página



• **Si no tiene instalado IIS:** ver detalle Elija la opción que dice 'Use ASP.NET Web Matriz Server'.
• **Si tiene instalado IIS:** Seleccione 'Use or Create an IIS Virtual Root'. Donde dice 'Application Name' Escriba 'WebMatrix'.

Nombre:

Password:

- Haga click en '**Start**'.

Va a ver en el explorador la ventana que aparece en la figura.

- Cierre la ventana del explorador de Internet.

Ahora vamos a poner código en el botón **Aceptar**.

- En el formulario, haga doble click en el botón **Aceptar**.
- Escriba el código siguiente:

```
Dim strNombre As String = TextBox1.Text
Dim strPassword As String = TextBox2.Text
'Para el ejemplo: El nombre de usuario debe ser: curso
'La password deber ser: vs
'chequeo de Nombre
If strNombre <> "curso" Then
    Response.Write("Error, Nombre de Usuario Incorrecto")
Else
    'chequeo de Password
    If strPassword <> "vs" Then
        Response.Write("Error, Password Incorrecta")
    Else
        'va otra la página
        Response.Write ("Hola !")
    End If
End If
```



- Ejecute la página. Haga distintas pruebas.
- Salve la página.

Hemos creado un formulario Web con controles usando WebMatrix.

4. 7. Revisión:

Es este capítulo hemos revisado conceptos, explorado ASP.NET, vimos como crear clases y componentes. También hemos creado páginas ASP.NET y usado controles. En el siguiente capítulo vamos a ver el acceso a los datos con ADO.NET